

# 漏洞报告 VULN-02: SQL注入

漏洞类型: SQL注入

测试小组: 天亮了么

被测小组: 天亮了么

测试时间: 2026-08-21

测试环境: 本机 localhost / 127.0.0.1:5001

测试账号: 课程测试账号, 未使用真实个人密码

## 1. 基本信息

本报告针对阶段二任务2, 仅在授权的本机课程靶场中完成。

## 2. 漏洞位置

URL/接口: gets.py 的 \_login、\_register

请求方法: GET/POST/DELETE (按接口实际方法)

关键参数: username、password、filename、file、csrf\_token (按任务适用)

所属功能: 登录/注册

## 3. 正常请求

先使用正常账号和合法参数完成对应功能, 再仅修改一个测试参数。请求记录和页面证据见附件目录。

正常请求示例: 在本机浏览器访问对应页面, 提交课程测试账号或合法文件; 不包含真实密码、Cookie 或 Token。

## 4. 漏洞复现步骤

1. 启动本机网盘并登录课程测试账号。
2. 按任务说明构造单一无害测试参数。
3. 发送请求并记录状态码、页面、文件或会话变化。
4. 使用修复后的同一参数再次验证。

## 5. 攻击效果或异常现象

使用 `username/password = ' OR '1'='1` 进行无害登录绕过探测, 返回401, 无登录跳转。

## 6. 漏洞原因分析

数据库查询已使用 %s 参数绑定, 未发现用户输入拼接 SQL。

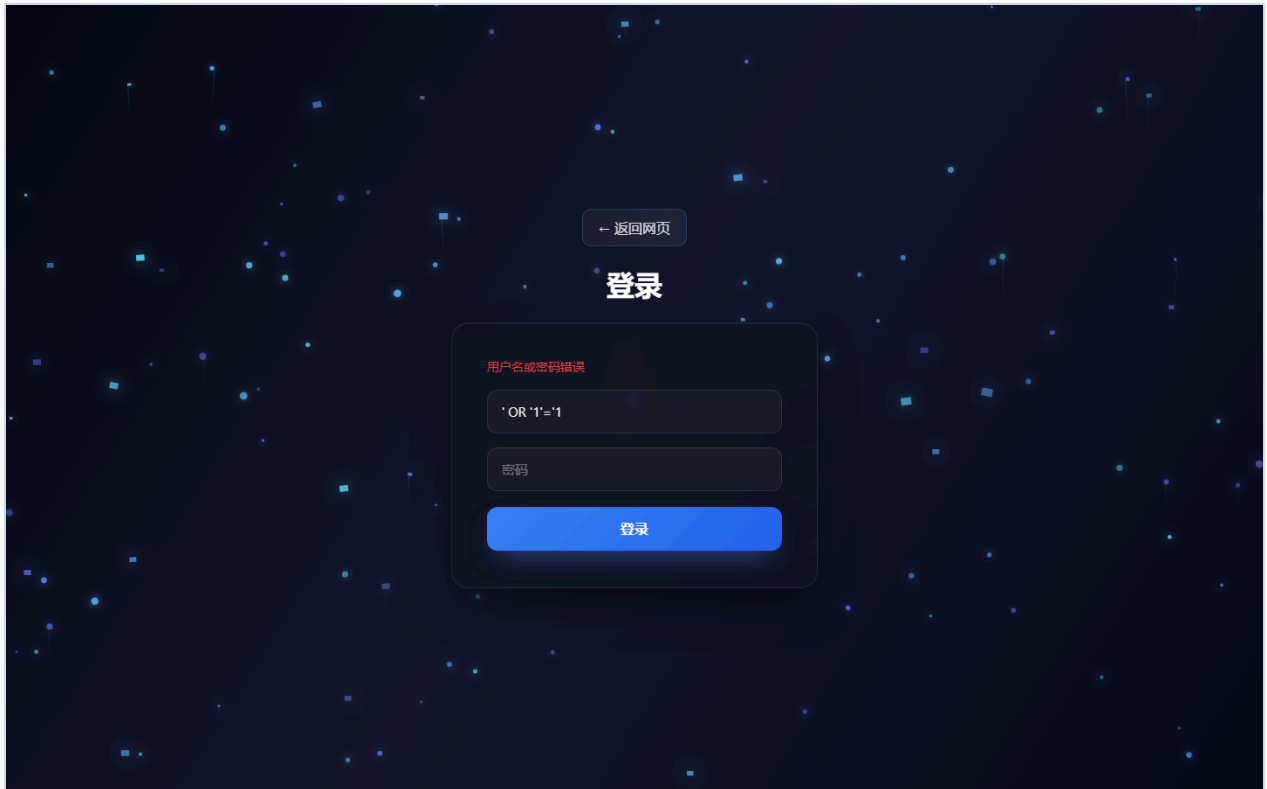
## 7. 影响范围

影响范围限于本机课程靶场及测试账号；不得外推为公网系统影响。具体后果取决于接口是否需要登录及资源归属。

## 8. 修复建议

保留参数化查询；新增数据库接口必须禁止字符串拼接并统一错误处理。

## 9. 附件清单



云 纯大子网盘

首页文件相册传输分享

搜索文件、图片、视频

登录注册纯

上传文件

全部文件

图片

视频

文档

收藏

回收站

空间使用

42.6 GB

已使用 42%，剩余空间充足

你的文件，随时可取

集中管理图片、视频、文档和学习资料，支持快速搜索、分类浏览和一键上传。

开始上传

新建文件夹

最近上传

8 个文件等待整理

我的分享

3 条链接正在生效

安全备份

今日已完成同步

最近文件

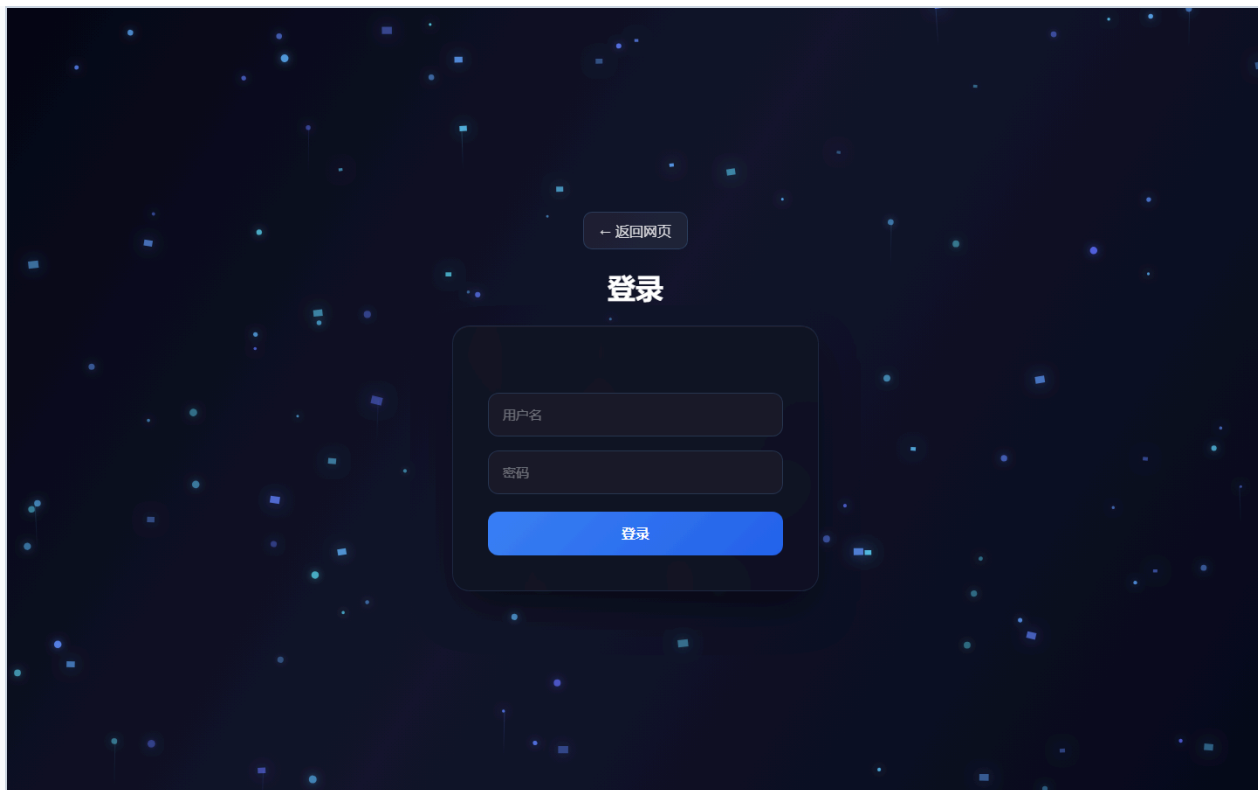
排序

筛选

| 名称                                                                                             | 类型  | 修改时间     | 大小     |
|------------------------------------------------------------------------------------------------|-----|----------|--------|
|  课程资料         | 文件夹 | 今天 16:40 | --     |
|  数据库作业说明.docx | 文档  | 昨天 21:18 | 1.8 MB |
|  首页设计参考.png   | 图片  | 5 月 16 日 | 3.4 MB |
|  项目演示视频.mp4   | 视频  | 5 月 15 日 | 128 MB |

Pretty-print ☐

```
{ "csrf_token": "bmZd77ZXj3Vvx7EXIZb4Ke2SKcberT8YhzcU5F1Gcotw" }
```



修复代码截图（实际源文件与行号）

## 任务 02 修复代码证据

gets.py:185-200

```
185     return jsonify(response), 401 if as_api else ("用户名或密码错误", 401)
186     conn = get_db_connection()
187     try:
188         with conn.cursor() as cursor:
189             cursor.execute("SELECT username, password FROM users WHERE username=%s", (username,))
190             user = cursor.fetchone()
191     finally:
192         conn.close()
193     valid = bool(user) and (
194         check_password_hash(user[1], password)
195         if user and user[1].startswith(("pbkdf2:", "bcrypt:"))
196         else bool(user) and user[1] == password
197     )
198     if not valid:
199         record_login_failure(username)
200     response = {"error": "用户名或密码错误"}
```

gets.py:285-299

```
285     conn = get_db_connection()
286     try:
287         with conn.cursor() as cursor:
288             cursor.execute("SELECT 1 FROM users WHERE username=%s", (username,))
289             if cursor.fetchone():
290                 return jsonify({"error": "用户名已存在"}), 409
291             cursor.execute(
292                 "INSERT INTO users(username, password) VALUES(%s, %s)",
293                 (username, generate_password_hash(password)),
294             )
295         conn.commit()
296     finally:
297         conn.close()
298     return jsonify({"success": True}), 200
299
```

